

Diseño de Infraestructura

PyME Financing Platform

Propósito del documento

Definir la infraestructura técnica recomendada para construir el sistema como una plataforma web full stack, alineada con el diseño de base de datos, la seguridad por roles, el manejo local de documentos, el análisis de riesgo y el matching financiero.

Decisión principal

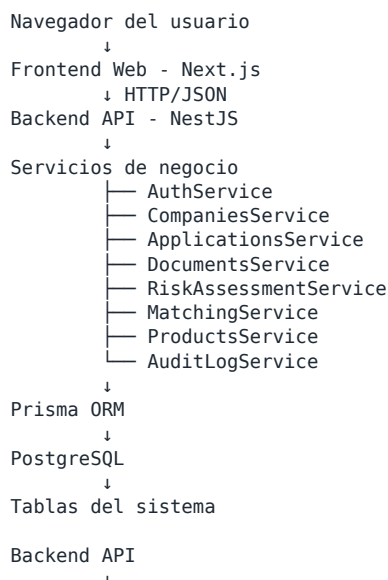
El sistema se implementará como una aplicación web. No se recomienda app móvil para el MVP, porque el objetivo es demostrar una plataforma fintech completa, con backend estructurado, base de datos relacional, documentos, reglas de negocio y dashboard operativo.

1. Resumen ejecutivo de infraestructura

La infraestructura recomendada es un monolito modular dividido en frontend, backend, base de datos y almacenamiento local de documentos. Esta arquitectura evita la complejidad de microservicios, pero mantiene separación clara de responsabilidades.

Capa	Tecnología	Uso dentro del sistema
Frontend web	Next.js + TypeScript	Interfaz web para operadores internos y solicitantes. Maneja formularios, tablas, expediente digital, carga de documentos y dashboards.
Backend API	NestJS + TypeScript	API modular con autenticación, autorización, validaciones, análisis de riesgo, matching financiero y auditoría.
Base de datos	PostgreSQL	Persistencia relacional para usuarios, empresas, solicitudes, documentos, productos, reglas, riesgo, recomendaciones e historial.
ORM	Prisma	Modelado del schema, migraciones, consultas tipadas y seeders.
Contenedores	Docker Compose	Orquestación local de frontend, backend, PostgreSQL, pgAdmin y volumen de uploads.
Archivos	Storage local /uploads	Carpeta local para PDFs e imágenes; en la base se guarda solo ruta y metadatos.
Auth	JWT + roles	Control de acceso con internal_operator y applicant.
API docs	Swagger / OpenAPI	Documentación y prueba de endpoints del backend.

2. Arquitectura general



```
/uploads/applications/{application_id}/...  
Documentos locales
```

El frontend no debe conectarse directamente a PostgreSQL ni acceder directamente a la carpeta de archivos. Toda operación debe pasar por la API para aplicar permisos, validaciones, auditoría y reglas de negocio.

3. Componentes principales

Componente	Responsabilidad principal	Notas de diseño
Frontend web	Presentar pantallas, formularios y resultados.	No debe decidir permisos críticos; solo mejora experiencia de usuario.
Backend API	Contener lógica de negocio y seguridad.	Debe validar rol, propietario, estado de solicitud y datos condicionales.
PostgreSQL	Guardar datos estructurados y relaciones.	Usar migraciones, restricciones, índices y enums definidos en el diseño de BD.
Prisma	Abstraer consultas y migraciones.	El schema Prisma debe reflejar fielmente el diseño de base de datos.
Uploads local	Guardar archivos cargados.	No guardar binarios en BD. Guardar ruta, hash, tipo MIME, tamaño y usuario.
Swagger	Exponer contratos de API.	Útil para mostrar profesionalismo y probar endpoints.

4. Infraestructura local con Docker Compose

Para desarrollo local se recomienda levantar todos los servicios con Docker Compose. Esto permite que el proyecto sea reproducible para GitHub y fácil de ejecutar en otra máquina.

```
docker-compose.yml  
├── postgres  
│   ├── Base de datos PostgreSQL  
│   └── Volumen persistente: postgres_data  
├── pgadmin  
│   └── Administración visual de PostgreSQL  
├── api  
│   ├── Backend NestJS  
│   ├── Prisma Client  
│   └── Montaje de volumen /uploads  
├── web  
│   └── Frontend Next.js  
└── volumes  
    ├── postgres_data  
    └── uploads_data
```

Servicio	Puerto sugerido	Descripción
web	3000	Frontend Next.js accesible desde navegador.
api	4000	Backend NestJS con endpoints REST y Swagger.
postgres	5432	Base de datos relacional.
pgadmin	5050	Panel opcional para revisar la base de datos.

5. Estructura recomendada del repositorio

Se recomienda un monorepo para mantener frontend, backend, base de datos, documentación y configuración en un solo repositorio. Esto facilita mostrar el proyecto en GitHub como una solución completa.

```
pyme-financing-platform/  
├── apps/  
│   └── web/
```

```
|
|   └─ Next.js frontend
└─ api/
    └─ NestJS backend

└─ packages/
    └─ database/
        ├── prisma/schema.prisma
        ├── migrations/
        └─ seed.ts

└─ uploads/
    └─ applications/
        └─ {application_id}/

└─ docs/
    ├── database-design.pdf
    ├── implementation-care.pdf
    └─ infrastructure-design.pdf

└─ docker-compose.yml
└─ README.md
└─ .env.example
```

6. Backend API - módulos recomendados

El backend debe ser un monolito modular. Cada módulo corresponde a una parte funcional del sistema y se alinea con las tablas del diseño de base de datos.

```
apps/api/src/modules/  
├── auth/  
├── users/  
├── roles/  
├── companies/  
├── applications/  
├── document-requirements/  
├── application-documents/  
├── financial-products/  
├── product-rules/  
├── risk-assessments/  
├── application-matches/  
├── status-history/  
└── audit-logs/
```

Módulo	Responsabilidad
auth	Login, JWT, guards, validación de usuario activo y rol.
companies	Registro y consulta de empresas; control por applicant_user_id.
applications	Creación, edición controlada y seguimiento de solicitudes.
application-documents	Carga local, metadatos, revisión documental y validación de archivos.
risk-assessments	Cálculo de riesgo, input_snapshot y creación de historial.
application-matches	Generación de recomendaciones ligadas a risk_assessment_id.
financial-products	Administración de productos financieros simulados.
product-rules	Reglas de compatibilidad y scoring.
audit-logs	Registro de acciones críticas sin exponer datos sensibles.

7. Frontend web - pantallas del MVP

Pantalla	Rol principal	Uso
Login	Todos	Autenticación de usuarios.
Dashboard	internal_operator	Vista general de solicitudes, estados y actividad.
Empresas	internal_operator / applicant	Gestión de empresas según permisos.
Solicitudes	internal_operator / applicant	Crear y consultar solicitudes de financiamiento.
Detalle de solicitud	Todos	Expediente digital con empresa, documentos, riesgo y recomendaciones.
Documentos	Todos	Carga, revisión y estado documental.
Análisis de riesgo	internal_operator	Ejecutar recálculo y consultar historial.
Matching financiero	internal_operator	Generar recomendaciones por análisis de riesgo.
Productos financieros	internal_operator	Crear, editar y desactivar productos.
Reglas de producto	internal_operator	Administrar reglas de compatibilidad.
Auditoría	internal_operator	Consultar acciones relevantes.

8. Flujo técnico de documentos

1. Usuario selecciona archivo en el frontend.
2. Frontend envía multipart/form-data al backend.
3. Backend valida permiso sobre la solicitud.
4. Backend valida extensión, MIME, tamaño y nombre.

5. Backend genera stored_filename seguro.
6. Backend guarda archivo en:
/uploads/applications/{application_id}/{stored_filename}
7. Backend calcula hash SHA-256.
8. Backend actualiza application_documents con ruta y metadatos.
9. Backend registra audit_log si aplica.
10. La descarga también pasa por backend para validar permisos.

Regla crítica

El frontend nunca debe construir rutas directas para descargar documentos. La descarga debe realizarse mediante endpoint protegido para validar que el usuario tenga permisos sobre la solicitud.

9. Seguridad y control de acceso

Riesgo	Medida de control
Un applicant ve datos de otro solicitante	Filtrar siempre por companies.applicant_user_id en backend.
Permisos solo ocultos en frontend	Aplicar guards y validaciones de rol en cada endpoint crítico.
Archivos descargados sin permiso	Servir archivos desde backend, no como carpeta pública abierta.
Auditoría con datos sensibles	No guardar password_hash, tokens ni contenido de documentos en audit_logs.
Estados inconsistentes	Implementar máquina de estados y rechazar transiciones inválidas.
Reglas de matching mal interpretadas	Convertir condition_value según rule_field antes de evaluar.

10. Entornos de ejecución

Entorno	Infraestructura	Propósito
Local	Docker Compose con web, api, postgres, pgAdmin y uploads.	Desarrollo y pruebas locales.
Demo	Frontend desplegado, backend desplegado, PostgreSQL administrado, storage externo opcional.	Mostrar proyecto en entrevista o portafolio.
Producción futura	Servicios administrados, HTTPS, backups, storage externo y monitoreo.	Escenario posterior al MVP.

Recomendación de despliegue futuro

En desarrollo está bien usar /uploads local. En producción conviene migrar archivos a un almacenamiento externo como S3, Supabase Storage, Cloudflare R2 o equivalente, para evitar perder documentos al redeplegar el backend.

11. Por qué no usar microservicios en el MVP

El sistema tiene varios módulos de negocio, pero todavía no requiere microservicios. Separarlo demasiado desde el inicio agregaría complejidad innecesaria en autenticación, red interna, despliegue, logs y comunicación entre servicios.

Decisión	Justificación
Monolito modular	Más simple de desarrollar, probar y desplegar. Mantiene estructura profesional por módulos.
Base de datos única	El dominio es relacional y requiere consistencia entre empresas, solicitudes, documentos, riesgo y matching.
Servicios internos	La separación debe hacerse en módulos y servicios de código, no en procesos independientes.

12. Flujo de operación del sistema

```

internal_operator / applicant inicia sesión
↓
Se registra o consulta empresa
↓
Se crea solicitud de financiamiento
↓
Se cargan documentos locales
↓
Operador revisa documentos
↓
Solicitud pasa a ready_for_analysis
↓
Operador ejecuta análisis de riesgo
↓
Se guarda risk_assessment + input_snapshot
↓
Se ejecuta matching financiero
↓
Se crean application_matches por risk_assessment_id
↓
Se consulta expediente digital

```

13. Orden recomendado de implementación

Fase	Implementación	Resultado esperado
1	Docker Compose, PostgreSQL, Prisma y migraciones.	Base del entorno funcionando.
2	Backend NestJS con auth, users y roles.	Login y JWT funcional.
3	Companies y financing_applications.	Empresas y solicitudes con permisos.
4	Catálogos: document_requirements, financial_products, product_rules.	Datos semilla listos.
5	Carga local de documentos.	Archivos guardados con metadatos y permisos.
6	Máquina de estados.	Transiciones controladas.
7	Risk assessments.	Cálculo con input_snapshot e historial.
8	Application matches.	Recomendaciones por análisis.
9	Frontend Next.js.	Pantallas principales integradas.
10	Audit logs, Swagger y README.	Entrega profesional para GitHub.

14. Cierre de diseño de infraestructura

Conclusión

La infraestructura propuesta es suficiente para implementar un MVP profesional: sistema web, backend modular, PostgreSQL, Prisma, Docker Compose, almacenamiento local de documentos, autenticación por JWT, roles y documentación de API. El siguiente paso es convertir este diseño en configuración real del repositorio, docker-compose.yml, schema Prisma y estructura base de frontend/backend.

Anexo de Actualización

PyME Financing Platform

Documento base conservado: Diseño de Infraestructura

Este anexo no reemplaza ni resume el documento base. El contenido original se conserva íntegro. Las reglas de este anexo se agregan al diseño y prevalecen cuando una regla anterior sea ambigua o use lenguaje de sugerencia.

A. Decisiones confirmadas para todos los documentos

Decisión	Instrucción obligatoria
Registro de applicant	El applicant debe poder crear su propia cuenta desde una ruta pública usando, como mínimo, fullName, email y password. El sistema debe asignar el rol applicant automáticamente.
Resultado visible para applicant	El applicant no debe ver el análisis interno completo, risk_score, pesos de reglas, DSCR detallado, audit_logs ni comparación interna de productos. Solo debe ver la decisión final publicada por el operador.
Operación interna	El internal_operator debe ver risk_assessments, application_matches, razones internas, porcentajes, indicadores financieros y debe crear/publicar la decisión final.
Corrección de necesidad	El internal_operator debe poder validar o corregir la necesidad declarada por el applicant mediante validated_need_type antes de generar matching o decisión final.
Documentos rechazados	Cuando el applicant suba nuevamente un documento rechazado, el sistema debe reemplazar el archivo en el mismo application_document, actualizar metadatos, cambiar status a uploaded y registrar audit_log.
Auditoría	El sistema debe auditar solo acciones sensibles; no debe registrar absolutamente todos los cambios ni guardar secretos, tokens, password_hash o contenido de documentos.

B. Regla de redacción para implementación

Toda frase del documento base que use “recomendado”, “sugerido”, “podría” o “si se implementa” debe interpretarse como instrucción obligatoria cuando describa una regla del MVP, una validación de seguridad, una entidad de base de datos, un endpoint crítico, una transición de estado o una prueba mínima.

C. Ajuste de infraestructura

No se agrega ningún servicio nuevo de infraestructura. El sistema debe seguir como monolito modular con frontend Next.js, backend NestJS, PostgreSQL, Prisma y almacenamiento local de archivos.

Elemento	Decisión actualizada
Backend	Agregar módulo application-decisions dentro del mismo backend NestJS. No crear microservicio.
Base de datos	Agregar migración para application_decisions y campos validated_need_type en financing_applications.
Storage local	Mantener carpeta uploads. En reemplazos documentales actualizar metadatos y auditar; no duplicar checklist.
Swagger	Documentar nuevos endpoints de registro público, validación de necesidad y decisiones.
Docker Compose	No agregar servicios. Mantener postgres, pgadmin, api y web según diseño base.